

Digital Twin Transaction Processing

Digital Twin supports transaction processing through both REST APIs and Kafka events, allowing institutions to choose the integration model that best aligns with the operational characteristics of each payment flow.

APIs are typically used for real-time transaction processing where immediate authorization and response handling are required. This model is well suited for instant payment rails such as RTP and FedNow, as well as other transaction types that require immediate posting and balance updates.

Kafka events are commonly used for high-volume or batch-oriented processing scenarios where transaction sequencing and timing must be controlled. This model is often used for ACH processing and other workflows where transactions may be grouped, ordered, or processed in near real time rather than individually synchronized request/response interactions.

By supporting both API-driven and event-driven processing models, Digital Twin allows institutions to optimize transaction processing behavior based on the speed, volume, and operational requirements of different payment rails and transaction types.

Credit and debit requests made via APIs or Kafka Events



Kafka Events

Used for batch-type transactions where order and timing are controlled

Use Cases:

- ACH transactions
- Posting credits ahead of debits
- Near real-time - ideal for high volume transactions



API

Used for real-time transactions requiring immediate processing

Use Cases:

- RTP
- FedNow
- Other immediate postings required

Data to Support Transaction Processing

Digital Twin transaction requests contain a standardized set of data elements used to support real-time authorization, balance management, and transaction processing across multiple payment types and rails.

Each request includes an entry kind, which defines the operational purpose of the transaction, such as a ledger posting, balance block, unblock operation, or reversal. The request also specifies an operation type indicating whether the transaction should be processed as pending or posted.

Transaction type and transaction code identify the business context of the transaction, including the associated payment rail and whether the transaction represents a debit or credit. Examples may include ACH, RTP, or FedNow transactions. Additional information such as the transaction amount, fulfillment status, and effective transaction timing are also included as part of the request.

To associate the transaction with the correct customer relationship, the request includes the account and routing number. A correlation ID is also included to identify the originating system or request context, enabling transaction traceability across integrated systems and payment flows.

This standardized transaction model allows Digital Twin to consistently process transactions across different payment rails, authorization scenarios, and operational workflows while maintaining flexibility for real-time and event-driven processing models.

Data required to send transaction request

Entry Kind What the transaction does (LEDGER, BLOCKING, UNBLOCKING, REVERSAL)	Operation Type Transaction status (PENDING or POSTED)
Transaction Type & Code References system config defining payment rail (ACH, RTP, FedNow) and debit/credit	Amount Transaction value
Fulfillment Status How much completed (PARTIAL for debits, TOTAL for credits)	Account & Routing # Links transaction to specific account
As-of Date Transaction timing	Correlation ID Identifier of the system that requested the transaction

Digital Twin supports multiple **entry kinds** that define how transactions should impact balances and account activity within the platform. Each entry kind represents a different type of operational behavior used during authorization and transaction processing.

- Ledger entries represent actual money movement, such as deposits, withdrawals, payments, and fees. Ledger entries are the only entry kind that support both pending and posted operation types. This allows transactions to be initiated before final settlement or confirmation occurs.

- Blocking entries are used to place holds against available balances. Common examples include debit card authorizations or check holds. Blocking transactions take effect immediately and are always processed as posted entries.
- Unblocking entries release previously placed holds and restore funds back to the available balance. Like blocking entries, unblocking transactions are processed immediately and always use a posted operation type.
- Reversal entries are used to correct or undo previously processed transactions, such as disputes, returned payments, or operational errors. Reversals are processed immediately as posted entries and typically reference a reverse transaction code associated with the original transaction.

This entry kind model allows Digital Twin to support real-time authorization, temporary holds, settlement workflows, and transaction corrections while maintaining precise control over how balances are impacted throughout the transaction lifecycle.

Entry Kind fields

LEDGER	Records actual money movement (deposits, withdrawals, payments, fees) <i>Can be PENDING (initiated) or POSTED (confirmed). Only entry kind where PENDING operation type is an option</i>
BLOCKING	Places hold that reduces available balance (debit card auth, check hold) <i>Takes effect immediately - always POSTED. Pending operation type is NOT an option</i>
UNBLOCKING	Releases previously placed hold, restoring available balance <i>Takes effect immediately - always POSTED. Pending operation type is NOT an option</i>
REVERSAL	Corrects or undoes previous transaction (disputes, errors, returned payments) <i>Takes effect immediately - always POSTED. Pending operation type is NOT an option Uses reverse transaction code paired with original transaction code.</i>

Pending & Posting

Digital Twin supports configurable transaction processing states that distinguish both the operational status of a transaction and the degree to which the transaction has been fulfilled.

Transactions may exist in either a pending or posted state. A pending transaction has been initiated but is still awaiting final confirmation or settlement. A posted transaction represents a finalized and confirmed entry that has completed processing. Pending status is supported only for ledger entries, allowing institutions to manage authorization scenarios where transactions may be approved before final settlement occurs. Other transaction types, such as blocking, unblocking, and reversal operations, are always processed as posted entries.

Digital Twin also supports fulfillment tracking to indicate whether a transaction was processed in full or only partially completed. Partial fulfillment is commonly associated with debit scenarios,

such as NSF or overdraft situations where only a portion of the requested amount can be processed. Total fulfillment indicates the full transaction amount was completed successfully. Credits are always processed as total transactions, following an all-or-nothing model.

This flexible transaction processing model allows Digital Twin to support real-time authorization, settlement timing differences, partial debit handling, and complex payment scenarios across multiple payment rails and transaction types.

Operation type and fulfillment status

Operation Type	Fulfillment Status
PENDING Transaction initiated but awaiting confirmation	PARTIAL Only part of transaction amount processed (debits only)
POSTED Transaction finalized and confirmed	TOTAL Full transaction amount completed (debits and credits)
Note: Only LEDGER entries can be PENDING. All other entry kinds (BLOCKING, UNBLOCKING, REVERSAL) are always POSTED.	Debits: Can be PARTIAL or TOTAL (e.g., PARTIAL common in NSF scenarios) Credits: Always TOTAL (all-or-nothing)

Holds

Digital Twin supports multiple types of holds that allow financial institutions to restrict balances or transaction activity as part of authorization, risk management, or regulatory workflows.

A **balance hold** is used to reserve or restrict a specific amount within an account balance. These holds may be configured as fixed amounts or ranges and are commonly used in legal, operational, or compliance-related scenarios where a portion of funds must remain unavailable for use.

A **transaction hold** is used to temporarily restrict specific transaction activity on an account, such as incoming credits or outgoing debits. These holds are often applied during fraud review, sanctions screening, or AML processing before funds are made available to the customer.

By supporting both balance-level and transaction-level holds, Digital Twin enables institutions to implement flexible operational controls while maintaining real-time authorization and balance management across different payment and compliance scenarios.

Example Real-Time Payment Flow with Digital Twin

This example below illustrates how Digital Twin supports real-time payment processing for inbound RTP transactions while maintaining continuous authorization and balance management.

The payment flow begins when the payment rail sends an inbound pacs.008 payment message to the institution's Payment Network Gateway (PNG). The PNG translates the request into a Digital Twin-native authorization request and submits it through the channel integration layer.

Digital Twin evaluates the transaction in real time using configured authorization rules, account status, limits, and balance logic. If approved, Digital Twin immediately updates the available balance by creating a pending credit entry and generates a success event. If the transaction is rejected, the rejection response must be communicated back through the network gateway to the payment rail.

Once authorization succeeds, the Payment Network Gateway responds to the payment rail with a pacs.002 acknowledgment message. After final network settlement occurs, the payment rail sends final confirmation back to the gateway, which then instructs Digital Twin to convert the transaction from pending to posted status.

Digital Twin generates a final success event that is consumed by the integration layer, which subsequently instructs the Core to record the finalized credit posting. If settlement fails at the payment network level, Digital Twin must instead remove the pending transaction so no posting occurs within the Core.

This architecture allows Digital Twin to manage real-time authorization and balance visibility independently of the Core while still ensuring finalized transactions are ultimately reflected within the institution's system of record.

Example RTP Receive with Digital Twin

